When one requirement changes, which parts should need to change?

Student notes for lecture 5. One rule change, the longest booking from 120 to 90 minutes, followed through two designs of Campus Rooms. Each scene shows a situation, asks you to predict, reveals the mechanism, changes one condition, compares the outcomes and names the principle: identical screens over different designs, a change spreading through copies, cohesion, an interface contract with its failure behavior, dependency direction, refactoring under checks, the cost of abstraction and a decision record.

Scenes in this lecture

#	SCENE (STUDENT HEADING)	ROLE	DEEP LINK
01	Two identical screens. Which design is easier to change?	Bridge	`week-05.html#/same-behavior-different-design`
02	Watch a single policy change spread	Anchor	`week-05.html#/dependency-example`
03	A module called Utilities. Do its contents belong together?	Bridge	`week-05.html#/cohesion-and-responsibility`
04	Same name, same parameters. Does the replacement fit?	Anchor	`week-05.html#/interface-contract`
05	Should the booking rule know how the database stores rows?	Bridge	`week-05.html#/dependency-direction`
06	Change the structure without quietly changing the promise	Anchor	`week-05.html#/refactoring-under-checks`
07	A flexible design can still be the wrong amount of design	Bridge	`week-05.html#/abstraction-cost`
08	A future maintainer needs the reason, not only the diagram	Bridge	`week-05.html#/design-rationale`

Campus Rooms is a fictional room-booking service; every value is a teaching example. The code is Python-like pseudocode.

Exam weeks need shorter bookings. How many places must change?

Room staff ask for one temporary change to Campus Rooms: the longest booking drops from 120 to 90 minutes. Before editing anything, find where that rule lives in the code.

The change: R-02 upper limit, temporary variant

The longest booking drops from 120 to 90 minutes; the shortest stays 30.

Campus Rooms today, and a search for "120"

COMPONENT	LINE	SEARCH	THE RULE?
Browser UI	`if minutes > 120: show_error("Up to 120 minutes")`	match	rule copy
Booking Handler	`if minutes < 30 or minutes > 120: return Rejected("duration")`	match	rule copy
Reminder	`if 30 <= minutes <= 2 * 60: schedule_reminder(booking)`	–	rule copy, missed (the same rule, written as 2 * 60)
Notification	`retry_after_seconds = 120`	match	no: a different 120: seconds before a retry
Room Catalog	`rooms = ["C101", "C202"]`	–	–
Storage	`bookings = load_bookings(room)`	–	–

3 copies of one rule: the search found 2, missed 1, and flagged 1 line that is a different 120.

The question of this week

When one requirement changes, which parts should need to change? (A) Every component that mentions it. (B) Whatever a search finds. (C) Only the part that owns the decision.

Four questions this lecture answers

- Can two identical screens hide different costs of change? (scenes 1–2)
- What belongs together in one module? (scene 3)
- What must a replacement keep, besides its name? (scenes 4–5)
- How do you restructure safely, and how much is enough? (scenes 6–8)

What this lecture builds

One rule change, R-02 from 120 to 90 minutes, followed through the code of Campus Rooms, and the design decisions around it.

What you already know, and where this lecture goes

When one requirement changes, which parts should need to change?

From Week 4

Carried forward: One prioritized change: changing a booking policy without causing widespread edits.

The components of Campus Rooms

- Browser UI: the booking screens people use.
- Booking Handler: receives a booking request and answers it.
- Reminder: reminds people before a booking starts.
- Policy: owns the booking rules, in one of the two designs.
- Room Catalog: knows the rooms: C101 and C202.
- Notification: sends confirmations and other messages.
- Storage: keeps the bookings.

Rules this lecture uses

- R-01: No overlapping confirmed bookings for the same room; 10:00 to 11:00 and 11:00 to 12:00 only touch.
- R-02: A booking lasts 30 to 120 minutes, both included.
- R-03: You may cancel your own booking before it starts.
- R-04: A failed notification never undoes a confirmed booking.

By the end of this lecture you can

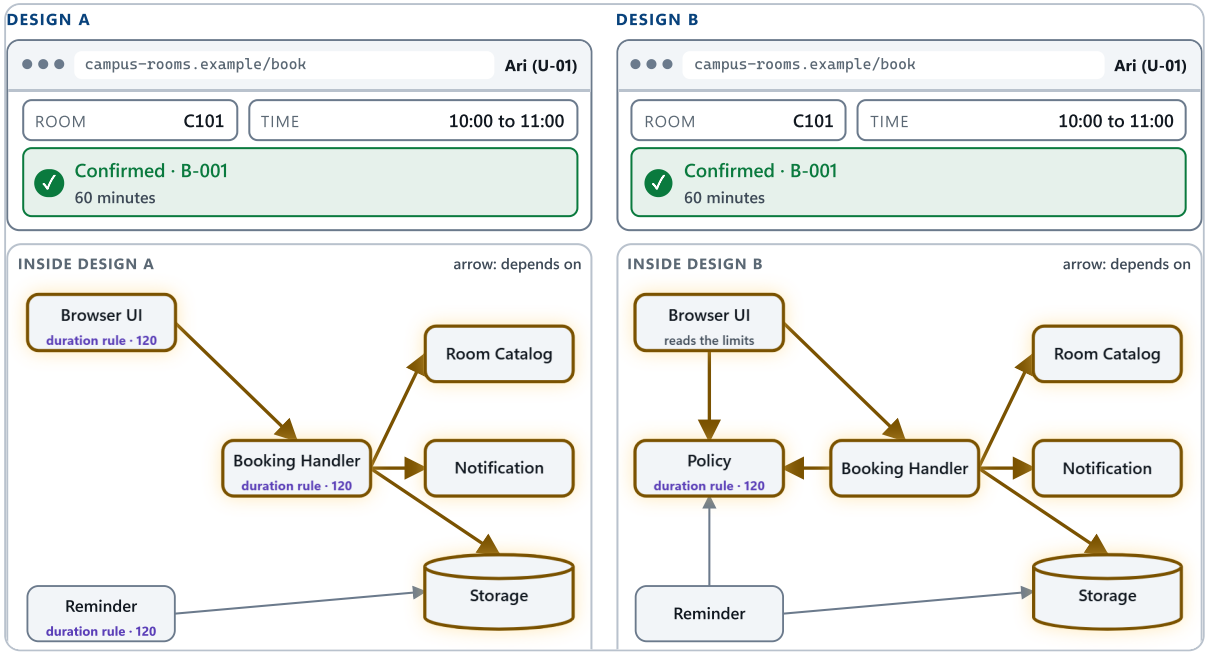
1. Trace the effect of a requirement change through dependent components.
2. Distinguish cohesive responsibility from unrelated work gathered in one module.
3. Explain how an interface can hide changeable implementation details.
4. Recognize why fewer lines or more abstractions do not automatically mean better design.
5. Explain refactoring as structural change with preserved intended behavior.

Two identical screens. Which design is easier to change?

Panel 1 of 2 · Two designs behind the same screen: one request through each

Condition: **Baseline: the same booking, C101 10:00 to 11:00, in two designs** State: **5 of 5**

Same behavior, different structure. Nothing on the screen tells them apart.



Both screens show the same behavior: Ari (U-01) books C101 from 10:00 to 11:00 and receives Confirmed B-001. Design A: Browser UI checks the duration itself; Booking Handler checks the duration again and decides; Room Catalog confirms that C101 exists; Storage finds no overlap, stores B-001; Notification sends the confirmation; later the Reminder checks the duration itself before it schedules a reminder. Design B: Browser UI reads the limits from Policy, for early feedback only; Booking Handler asks Policy, then decides; Room Catalog confirms that C101 exists; Storage finds no overlap, stores B-001; Notification sends the confirmation; later the Reminder asks Policy before it schedules a reminder. Arrows point from a component to the one it depends on.

Principle · Design quality becomes visible through the changes and failures a structure must accommodate.

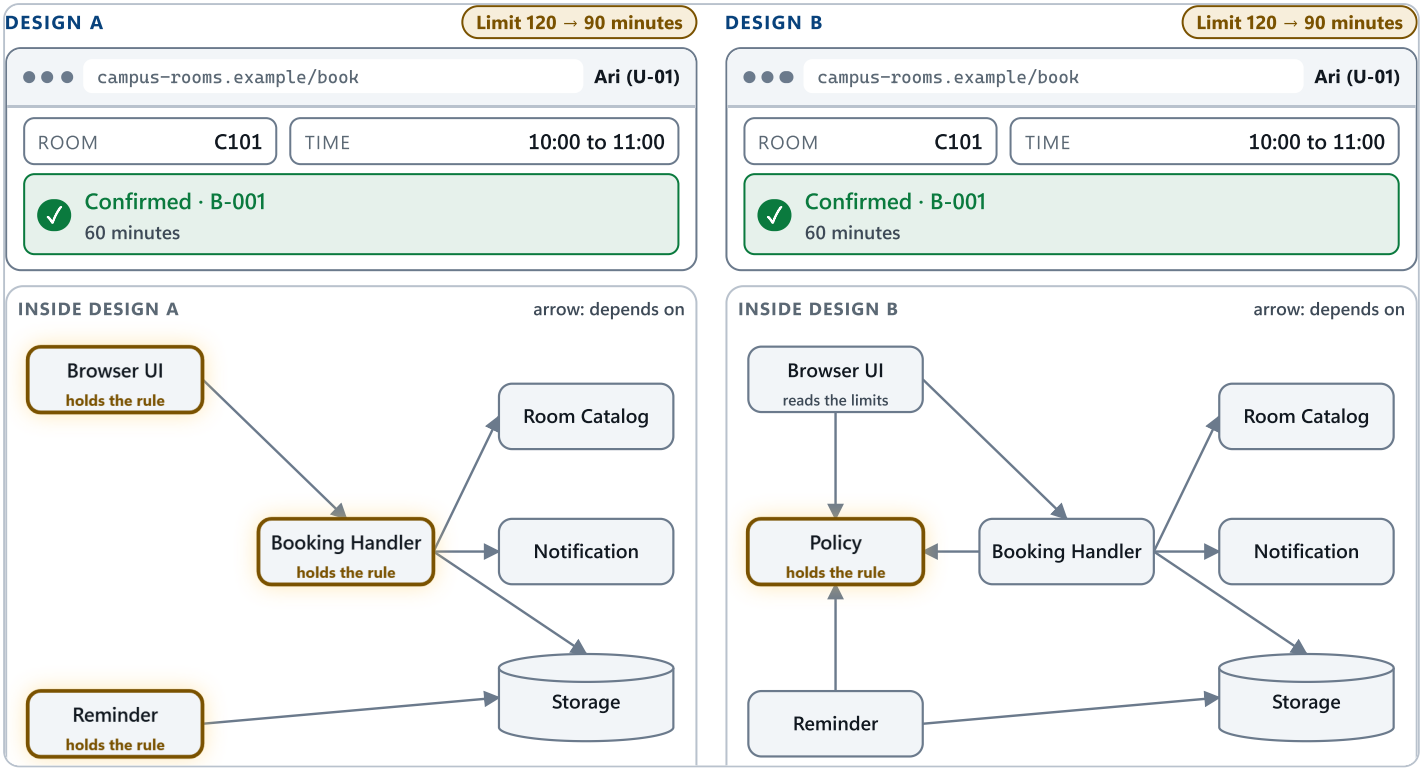
In short · Structure shows itself when something changes.

Two identical screens. Which design is easier to change?

Panel 2 of 2 · The limit drops to 90 minutes: where the rule lives

Condition: **Changed: the longest booking drops from 120 to 90 minutes; the screens and the goal stay the same** State: **3 of 3**

The screens still look the same. The cost of the change does not.



The screen and the user's goal stay the same. The duration rule lives in Browser UI, Booking Handler and Reminder in Design A, and only in Policy in Design B; the Browser UI of Design B reads the limits from Policy for early feedback. Scene 2 follows the change into each component.

Principle · Design quality becomes visible through the changes and failures a structure must accommodate.

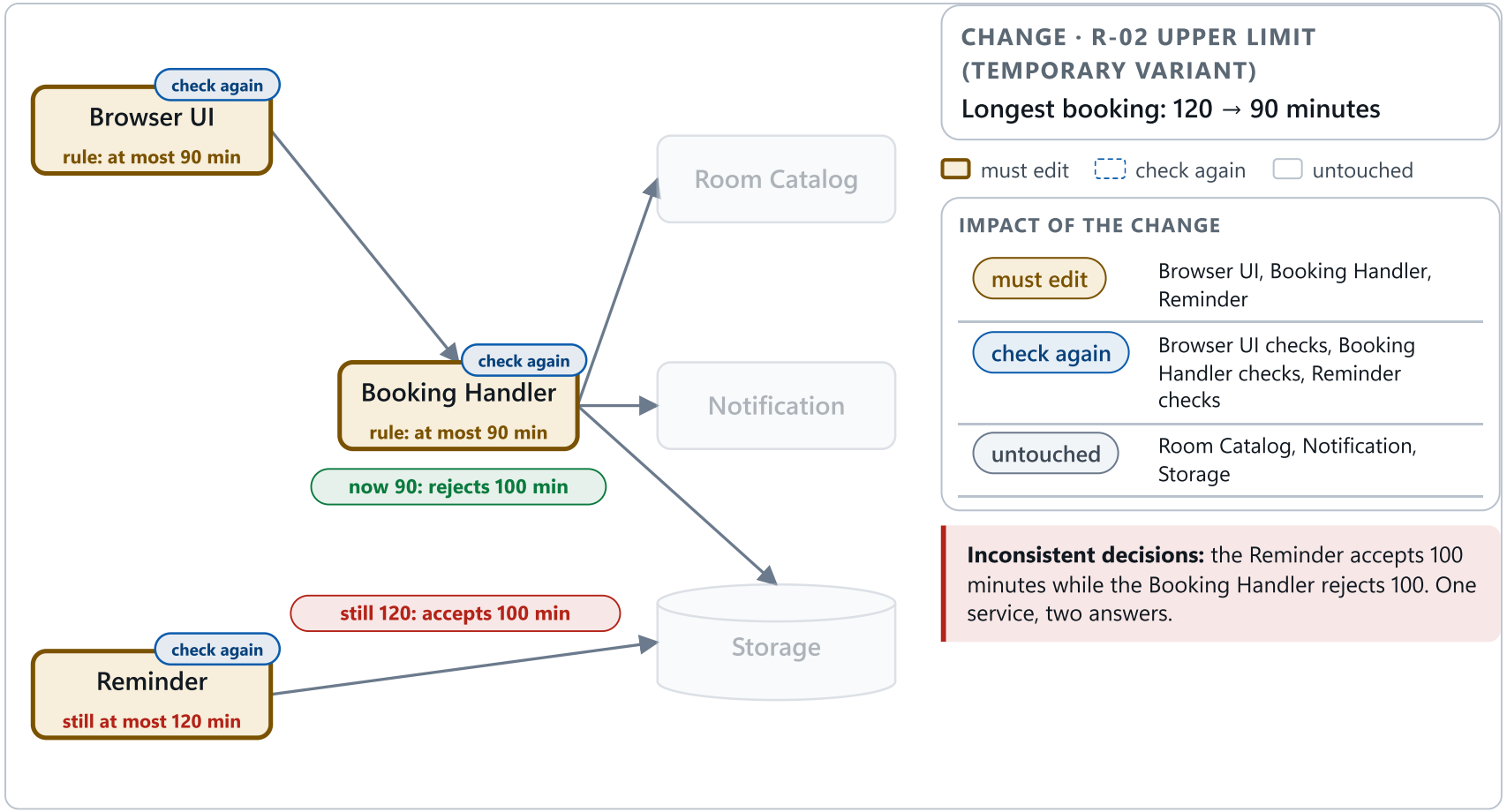
In short · Structure shows itself when something changes.

Watch a single policy change spread

Panel 1 of 3 · Design A after the change: 3 edits, 3 rechecks, one missed copy

Condition: **Baseline: Design A, the duration rule is copied into 3 components** State: **6 of 7**

One copy is missed. The **Reminder** still says 120, so it accepts a 100-minute booking that the Booking Handler now rejects.



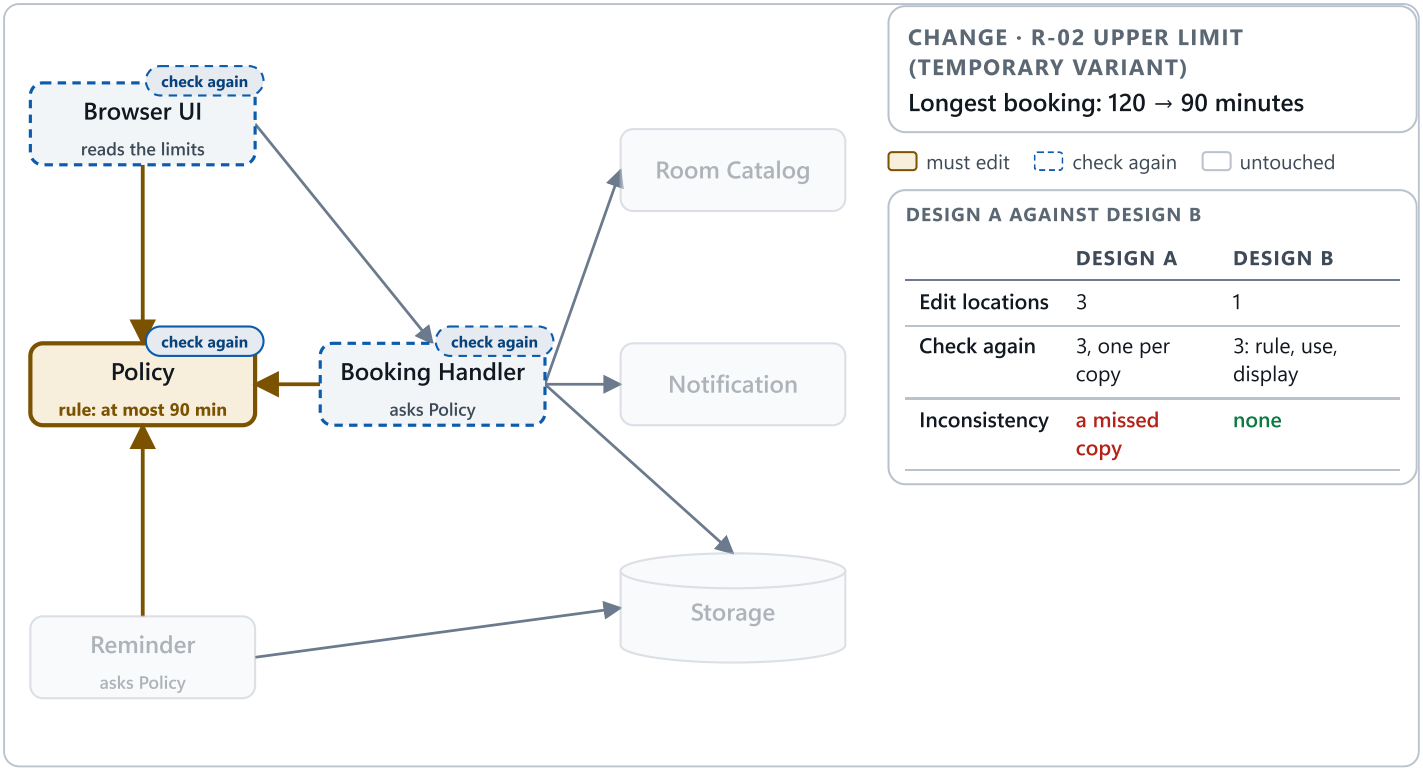
Legend: thick yellow border = must edit; dashed blue border with a pill = check again; faded = untouched. The Reminder still carries the old limit and accepts a 100-minute booking that the Booking Handler rejects 100: inconsistent decisions.

Watch a single policy change spread

Panel 2 of 3 · Design B after the same change: 1 edit, visible rechecks

Condition: **Changed: Design B, where one Policy component owns the rule** State: **7 of 7**

Policy keeps the decision in one place, and its arrows show you what to check again.



Must edit: Policy. Check again: Policy checks, Booking Handler integration check, Browser UI limit display. Untouched: Room Catalog, Notification, Storage, Reminder logic. One module does not mean only one file ever changes.

Principle · Useful boundaries localize decisions while keeping their dependents and evidence visible.

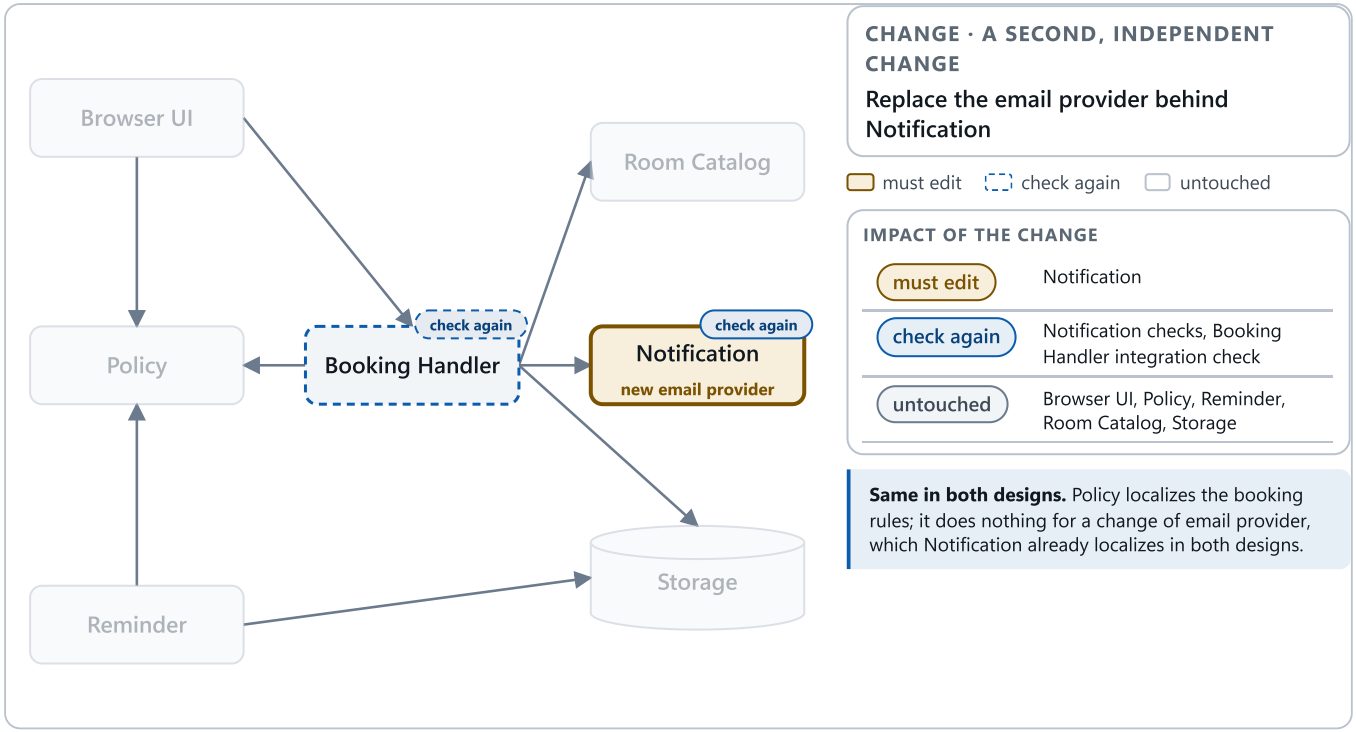
In short · Keep each decision in one place, and its users visible.

Watch a single policy change spread

Panel 3 of 3 · A second, independent change: the email provider

Condition: **Changed: a different change on the same designs: replace the email provider behind Notification** State: **5 of 5**

Each boundary localizes the decisions it was drawn around.



Replace the email provider behind Notification. Must edit: Notification. Check again: Notification checks, Booking Handler integration check. Untouched in Design B: Browser UI, Policy, Reminder, Room Catalog, Storage; Design A gives the same picture. Policy localizes the booking rules; it does nothing for a change of email provider, which Notification already localizes in both designs.

Principle · Useful boundaries localize decisions while keeping their dependents and evidence visible.

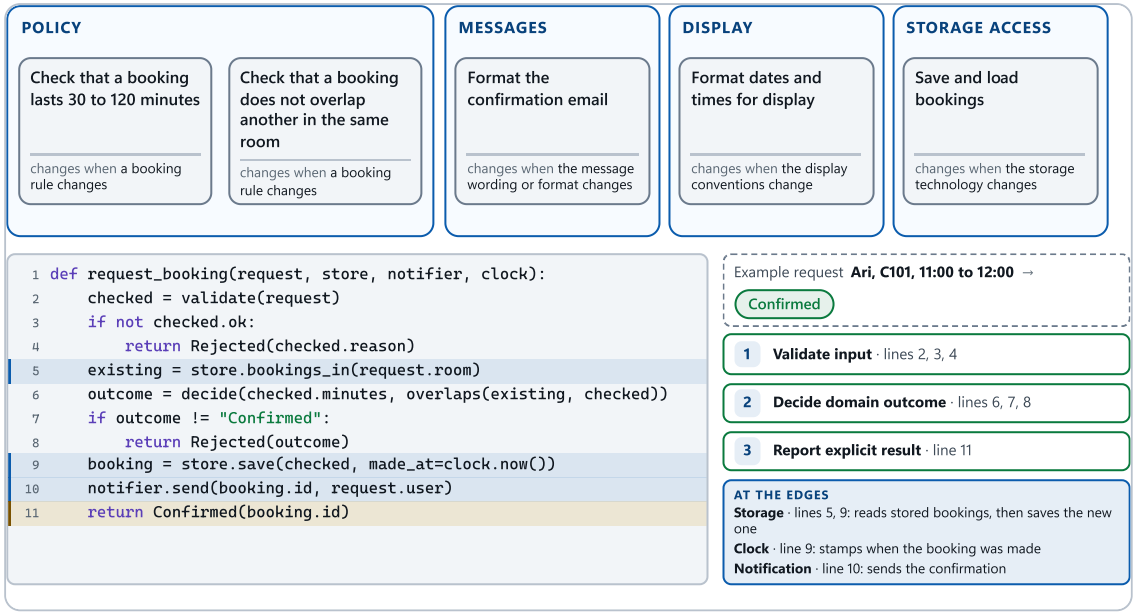
In short · Keep each decision in one place, and its users visible.

A module called Utilities. Do its contents belong together?

Panel 1 of 2 · Five tasks regrouped by reason to change, and one request built from them

Condition: **Baseline: one module, utilities.py, holds five unrelated tasks** State: **7 of 7**

The decision is small, pure and easy to find: that is the point of the grouping.



Check that a booking lasts 30 to 120 minutes: changes when a booking rule changes (Policy). Check that a booking does not overlap another in the same room: changes when a booking rule changes (Policy). Format the confirmation email: changes when the message wording or format changes (Messages). Format dates and times for display: changes when the display conventions change (Display). Save and load bookings: changes when the storage technology changes (Storage access). In the request, validate input (lines 2, 3, 4): the request is well formed: whole minutes, end later than start; decide domain outcome (lines 6, 7, 8): the booking rules R-02 and R-01 decide, reading nothing outside; report explicit result (line 11): the caller receives Confirmed or Rejected with a reason. At the edges: storage, reads stored bookings, then saves the new one; clock, stamps when the booking was made; notification, sends the confirmation.

Principle · Cohesion concerns meaningful related responsibility, not simply making modules small.

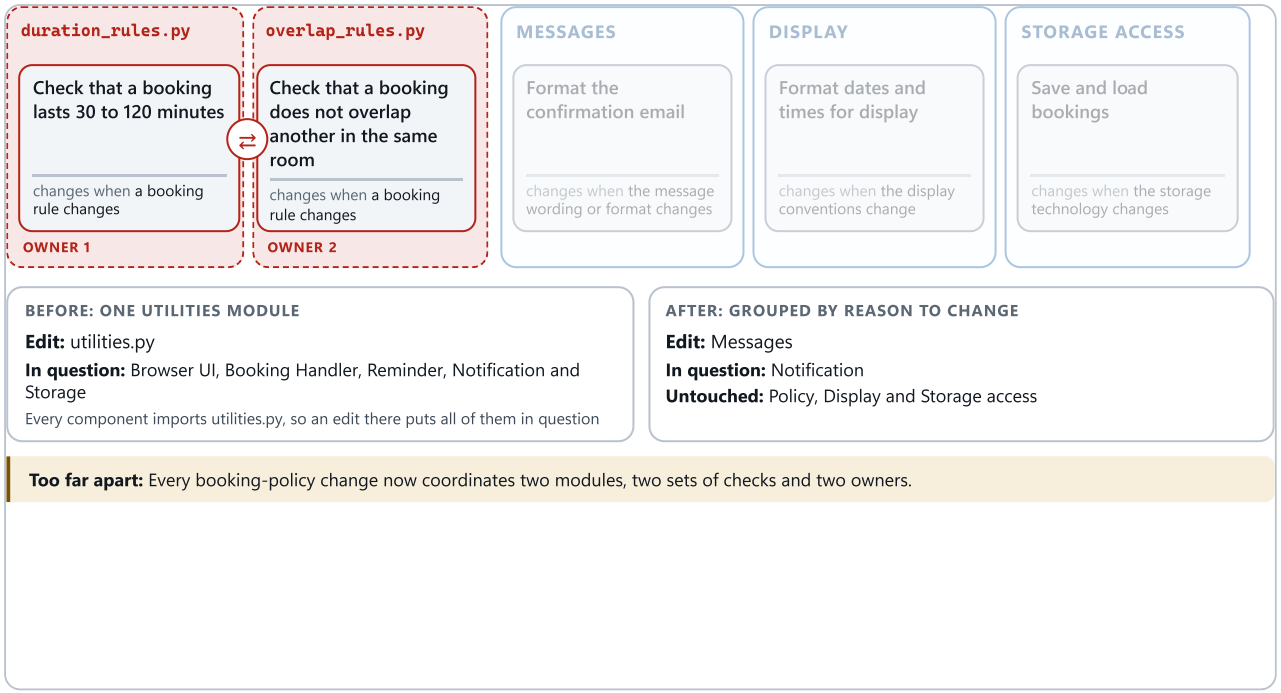
In short · Group what changes together; small alone is not the goal.

A module called Utilities. Do its contents belong together?

Panel 2 of 2 · A new message format, before and after the regrouping

Condition: **Changed: add a text-message format for confirmations; the booking rules stay the same** State: **5 of 5**

Draw boundaries around what changes together.



Add a text-message format for confirmations. Before: edit utilities.py; in question: Browser UI, Booking Handler, Reminder, Notification and Storage, because every component imports utilities.py, so an edit there puts all of them in question. After: edit Messages; in question: Notification; untouched: Policy, Display and Storage access. Splitting too far: the duration rule and the overlap rule in two separate modules with two owners; every booking-policy change now coordinates two modules, two sets of checks and two owners.

Principle · Cohesion concerns meaningful related responsibility, not simply making modules small.

In short · Group what changes together; small alone is not the goal.

Same name, same parameters. Does the replacement fit?

Panel 1 of 3 · The contract, and the current implementation that keeps it

Condition: **Baseline: the current implementation, Mail A (current)** State: **5 of 5**

Delivery is **at least once; duplicates possible**. The contract says so, so callers expect a second email after a retry.

THE INTERFACE

```
Notifier.send(booking_id, recipient) -> DeliveryStatus
```

PLUGGED IN **Mail A (current)** | **Mail B (replacement)**

PROMISE	THE CONTRACT SAYS	MAIL A
Call shape PRECONDITION	send(booking_id, recipient)	✓ the same call
Returns OUTCOME	a DeliveryStatus: Queued or Failed	✓ Queued or Failed
On failure ERROR EXPECTATION	reports Failed; never raises into the caller	✓ reports Failed
Confirmation SIDE EFFECT	the booking confirmation never waits for the email (R-04)	✓ never waits
Delivery SIDE EFFECT	at least once; a duplicate email is possible	✓ at least once; duplicates possible

A FAILED DELIVERY — B-001 · C101 10:00 to 11:00

1. **Storage:** B-001 is Confirmed
2. **Mail A:** returns Failed
3. **Booking Handler:** records the email as pending, to retry

campus-rooms.example/book — **Ari (U-01)**

✓ Booking confirmed: C101, 10:00 to 11:00.
Confirmation email: pending, we will retry.

R-04 kept: the booking stands, and the email is pending for a retry.

At least once: a retry after a lost answer can send the same email twice. The contract says so, and callers plan for it.

Notifier.send(booking_id, recipient) -> DeliveryStatus. Call shape (precondition): send(booking_id, recipient); Returns (outcome): a DeliveryStatus: Queued or Failed; On failure (error expectation): reports Failed; never raises into the caller; Confirmation (side effect): the booking confirmation never waits for the email (R-04); Delivery (side effect): at least once; a duplicate email is possible. With Mail A (current), a failed delivery: returns Failed; the handler records the email as pending, to retry; the screen says “Booking confirmed: C101, 10:00 to 11:00. Confirmation email: pending, we will retry.”

Same name, same parameters. Does the replacement fit?

Panel 2 of 3 · The replacement raises into the booking path

Condition: **Changed: the replacement, Mail B (replacement)** State: **5 of 5**

Ari tries again and R-01 rejects him: the new request overlaps B-001, his own stored booking. The user promise is broken.

THE INTERFACE

```
Notifier.send(booking_id, recipient) -> DeliveryStatus
```

PLUGGED IN **Mail B (replacement)**

PROMISE	THE CONTRACT SAYS	MAIL B
Call shape PRECONDITION	send(booking_id, recipient)	✓ the same name and parameters
Returns OUTCOME	a DeliveryStatus: Queued or Failed	✗ **returns None**
On failure ERROR EXPECTATION	reports Failed; never raises into the caller	✗ **raises into the booking path**
Confirmation SIDE EFFECT	the booking confirmation never waits for the email (R-04)	✗ **the exception stops the confirmation**
Delivery SIDE EFFECT	at least once; a duplicate email is possible	~ exactly one attempt, no retry

A FAILED DELIVERY B-001 · C101 10:00 to 11:00

1. **Storage:** B-001 is Confirmed
2. **Mail B:** raises an exception
3. **Booking Handler:** stops before it answers

campus-rooms.example/book **Ari (U-01)**

✗ Something went wrong. Please try again.

R-04 broken: the booking is stored, but Ari was told it failed.

What happens next: Ari tries again and R-01 rejects him: the new request overlaps B-001, his own stored booking.

Mail B (replacement): Call shape: fits, the same name and parameters; Returns: breaks, returns None; On failure: breaks, raises into the booking path; Confirmation: breaks, the exception stops the confirmation; Delivery: differs, exactly one attempt, no retry. A failed delivery: the booking is stored as Confirmed, Mail B raises an exception, the handler stops before it answers; the screen says “Something went wrong. Please try again.” Ari tries again and R-01 rejects him: the new request overlaps B-001, his own stored booking. This breaks R-04.

Same name, same parameters. Does the replacement fit?

Panel 3 of 3 · Only the failure behavior changes: the failure is reported

Condition: **Changed: only Mail B's failure behavior; it reports Failed instead of raising** State: **4 of 4**

Behavior and failure semantics are part of the interface.

THE INTERFACE

```
Notifier.send(booking_id, recipient) -> DeliveryStatus
```

PLUGGED IN **Mail B, failures reported**

PROMISE	THE CONTRACT SAYS	MAIL B, FAILURES REPORTED
Call shape PRECONDITION	send(booking_id, recipient)	✓ the same name and parameters
Returns OUTCOME	a DeliveryStatus: Queued or Failed	~ None, or Failed
On failure ERROR EXPECTATION	reports Failed; never raises into the caller	✓ reports Failed
Confirmation SIDE EFFECT	the booking confirmation never waits for the email (R-04)	✓ never waits
Delivery SIDE EFFECT	at least once; a duplicate email is possible	~ one attempt; the caller keeps the retry

A FAILED DELIVERY B-001 · C101 10:00 to 11:00

1. **Storage:** B-001 is Confirmed
2. **Mail B:** returns Failed
3. **Booking Handler:** records the email as pending, to retry

campus-rooms.example/book Ari (U-01)

✓ Booking confirmed: C101, 10:00 to 11:00.
Confirmation email: pending, we will retry.

R-04 kept: the booking stands, and the email is pending for a retry.

Mail B, failures reported: Call shape: fits, the same name and parameters; Returns: differs, None, or Failed; On failure: fits, reports Failed; Confirmation: fits, never waits; Delivery: differs, one attempt; the caller keeps the retry. The booking stays confirmed and the email is pending.

Principle · An interface contract includes behavior and failure semantics, not just matching names or parameter lists.

In short · Matching names are not matching behavior.

One booking request, three kinds of answer

The booking interface returns one of three results. Each one tells the caller something different, and a well-shaped value answers only one of four separate questions.

The booking interface

```
def book(request) -> Confirmed | Rejected | Unavailable
```

- Confirmed: the booking decision was made and stored; carries the booking id.
- Rejected: a domain rule said no; trying again unchanged gives the same answer; carries the rule and reason.
- Unavailable: the decision could not be made right now; nothing was decided; carries when to try again.

Four requests

- Ari, C101, 11:00 to 12:00 → Confirmed: 60 minutes, and 11:00 only touches B-001's end: adjacent, not overlapping (R-01).
- Bo, C101, 10:30 to 11:30 → Rejected: it overlaps B-001 from 10:30 to 11:00 (R-01).
- Ari, C202, 10:00 to 12:30 → Rejected: 150 minutes is over the 120-minute limit (R-02).
- Ari, C202, 14:00 to 15:00, while storage is down → Unavailable: the stored bookings could not be read, so no decision was made.

Four separate questions

1. The value has the right shape? Evidence: the type annotation, checked by a type checker. Example: Confirmed always carries a booking id.
2. The booking obeys the domain rules? Evidence: Policy checks for R-01 and R-02. Example: a well-shaped request for 150 minutes still breaks R-02.
3. The requester may do this? Evidence: a permission check (R-03). Example: Bo's well-shaped request to cancel B-001 is denied: he is not the owner.
4. The decision holds when requests race? Evidence: storage enforces one atomic booking decision (Week 6). Example: two overlapping requests can each pass a check before either is stored.

A well-shaped value answers the first question only. Domain rules, permission and concurrency each need their own check.

Should the booking rule know how the database stores rows?

Panel 1 of 3 · Policy reads the database rows: both views point the same way, and a storage change edits the rule

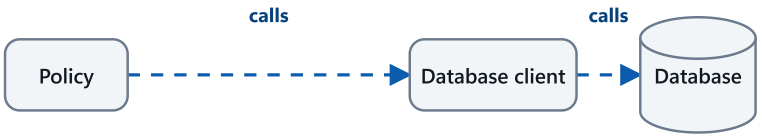
Condition: **Baseline: Policy reads the database client's rows** State: **4 of 4**

Now move the bookings to a different database product. **Policy must change**, although no booking rule changed.

SOURCE CODE: WHO DEPENDS ON WHOM — solid arrows



RUNTIME: WHO CALLS WHOM — dashed arrows



POLICY · THE OVERLAP RULE

```
1 def overlaps(room, start, end, db):
2     rows = db.query("SELECT start_min, end_min FROM bookings "
3                     "WHERE room = ?", room)
4     for row in rows:
5         if row[0] < end and start < row[1]:
6             return True
7     return False
```

WHAT THE RULE NOW KNOWS

- the query text
- the table and column names
- the position of each value in a row

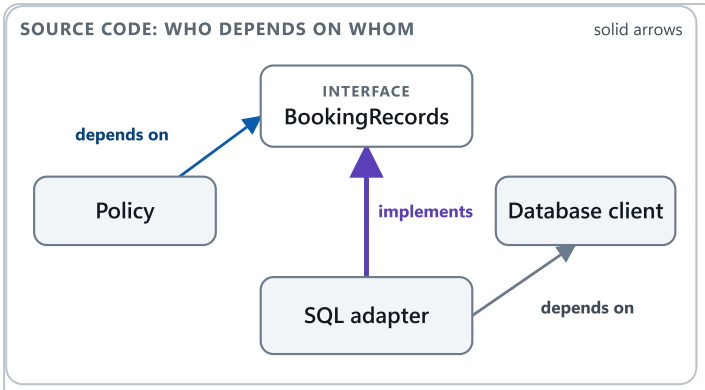
Source view: Policy depends on the database client. Runtime view: Policy calls the client, which calls the database. The rule code knows the query text, the table and column names and the position of each value in a row. When Policy must change because the rule code itself reads rows in the old database's shape, the rule is edited although no booking rule changed.

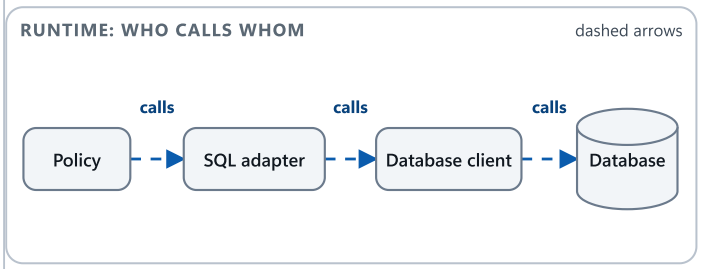
Should the booking rule know how the database stores rows?

Panel 2 of 3 · Policy depends on BookingRecords: the source and runtime arrows differ

Condition: **Changed: Policy depends on a small interface it owns, BookingRecords** State: **4 of 4**

Each part knows only what its responsibility needs.





POLICY · THE OVERLAP RULE

```python
class BookingRecords:              # what Policy needs
    def intervals_in(self, room):  # -> [(start, end)]
        ...

def overlaps(room, start, end, records):
    for s, e in records.intervals_in(room):
        if s < end and start < e:
            return True
    return False
```

SQL ADAPTER · STORAGE SIDE

```python
class SqlBookingRecords(BookingRecords):
    def intervals_in(self, room):
        rows = self.db.query(
            "SELECT start_min, end_min FROM bookings "
            "WHERE room = ?", room)
        return [(row[0], row[1]) for row in rows]
```

Source view (solid): Policy depends on BookingRecords; the SQL adapter depends on BookingRecords and on the database client. Runtime view (dashed): Policy calls the SQL adapter, which calls the client, which calls the database. The row shape lives only in the adapter.

Principle · Depend on the behavior a responsibility needs, and isolate details that change for other reasons.

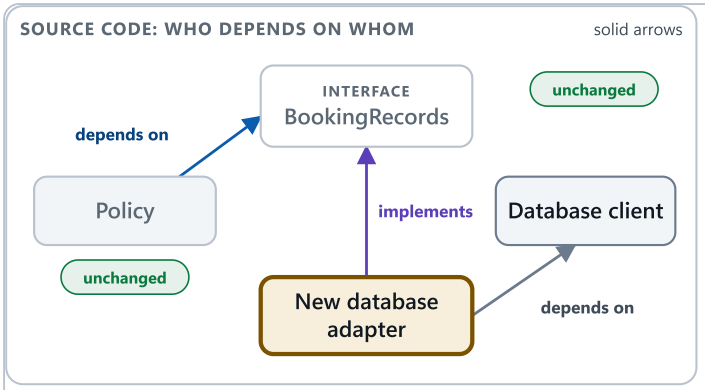
In short · Depend on what you need, not on how it is stored.

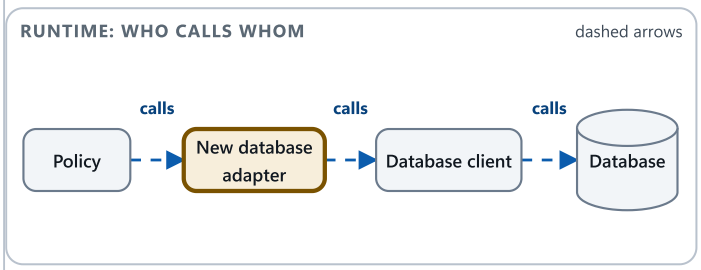
Should the booking rule know how the database stores rows?

Panel 3 of 3 · A different database product: a new adapter, the same Policy

Condition: **Changed: move the bookings to a different database product, with the interface in place** State: **4 of 4**

Isolating a detail moves the change; it does not remove the checks.





POLICY · THE OVERLAP RULE

```
class BookingRecords:              # what Policy needs
    def intervals_in(self, room):  # -> [(start, end)]
        ...

def overlaps(room, start, end, records):
    for s, e in records.intervals_in(room):
        if s < end and start < e:
            return True
    return False
```

STILL TO VERIFY

- The new adapter returns the same intervals for the same stored bookings
- Two overlapping requests that race still cannot both be confirmed: the atomic booking decision

Move the bookings to a different database product. Edit: New database adapter. Unchanged: Policy and BookingRecords. Still to verify: the new adapter returns the same intervals for the same stored bookings; two overlapping requests that race still cannot both be confirmed: the atomic booking decision.

Principle · Depend on the behavior a responsibility needs, and isolate details that change for other reasons.

In short · Depend on what you need, not on how it is stored.

Change the structure without quietly changing the promise

Panel 1 of 2 · Before and after step 1: the same six answers

Condition: **Baseline: extract one policy function in small steps** State: **5 of 5**

The structure changed and the observable answers did not. That is **refactoring**.

THE CODE — after step 1

```
1 def duration_ok(minutes):
2     return 30 <= minutes <= 120
3
4 def can_book(start, end):
5     return duration_ok(end - start)
6
7 def worth_reminding(start, end):
8     return duration_ok(end - start)
```

STEP LOG

before	two copies of the check	**6 of 6 match**
step 1	extract duration_ok	**6 of 6 match**
step 1	the Reminder calls it too	**6 of 6 match**

In practice: editors such as VS Code and IntelliJ IDEA perform small refactorings like extracting a function for you; the six examples still run after each one.

SIX EXAMPLES · REPLAYED AFTER EVERY STEP

MINUTES	R-02 PROMISE	CODE SAYS
29	no	✓ **no**
30	yes	✓ **yes**
60	yes	✓ **yes**
119	yes	✓ **yes**
120	yes	✓ **yes**
121	no	✓ **no**

✓ All six answers match the promise.

Before: can_book and worth_reminding each carry the check (the second written as 2 * 60). Step 1 extracts duration_ok(minutes) and both call it. The six examples 29, 30, 60, 119, 120, 121 answer no, yes, yes, yes, yes, no before and after: the promise of R-02.

Principle · Refactoring should preserve intended observable behavior; small steps and relevant checks make deviations easier to locate.

In short · Change the structure in small steps; keep the answers.

Change the structure without quietly changing the promise

Panel 2 of 2 · The slip: 120 minutes changes its answer, and step 3 restores it

Condition: **Changed: one step retypes the bound as minutes < 120** State: **3 of 5**

Replay: **120 minutes** now answers no, but the promise says yes. The boundary example catches the slip.

THE CODE — step 2

```
1 def duration_ok(minutes):
2     return 30 <= minutes < 120
3
4 def can_book(start, end):
5     return duration_ok(end - start)
6
7 def worth_reminding(start, end):
8     return duration_ok(end - start)
```

STEP LOG

after step 1	both callers use duration_ok	6 of 6 match
step 2	<= retyped as <	5 of 6 match

SIX EXAMPLES · REPLAYED AFTER EVERY STEP

MINUTES	R-02 PROMISE	CODE SAYS
29	no	✓ no
30	yes	✓ yes
60	yes	✓ yes
119	yes	✓ yes
120	yes	✗ no
121	no	✓ no

✗ 120 minutes: the answer changed.

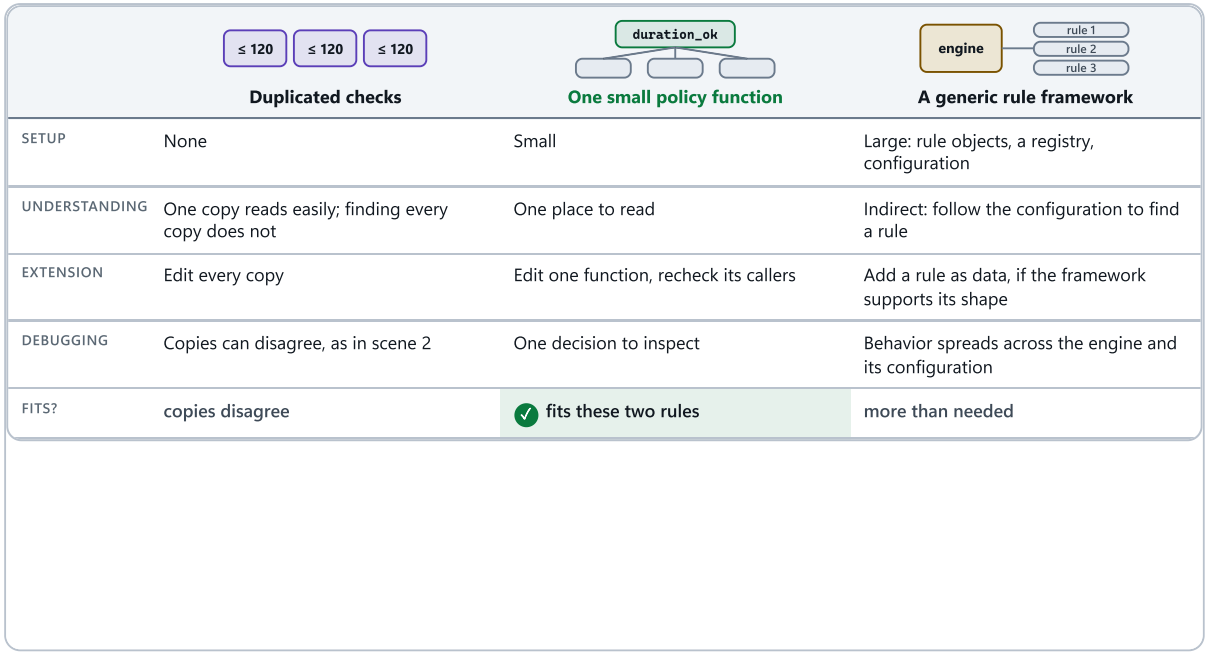
Step 2 retypes the bound as minutes < 120. Replayed, 120 minutes answers no where the promise says yes; step 3 restores <= and all six match again (yes for 120).

A flexible design can still be the wrong amount of design

Panel 1 of 3 · Two rules, a small team: one small policy function fits

Condition: **Baseline: two clear policy rules (R-01 and R-02) and a small team** State: 6 of 6

For two rules and a small team: **one small policy function**.



	Duplicated checks	One small policy function	A generic rule framework
SETUP	None	Small	Large: rule objects, a registry, configuration
UNDERSTANDING	One copy reads easily; finding every copy does not	One place to read	Indirect: follow the configuration to find a rule
EXTENSION	Edit every copy	Edit one function, recheck its callers	Add a rule as data, if the framework supports its shape
DEBUGGING	Copies can disagree, as in scene 2	One decision to inspect	Behavior spreads across the engine and its configuration
FITS?	copies disagree	✓ fits these two rules	more than needed

Setup: duplicated checks, none; one small policy function, small; a generic rule framework, large: rule objects, a registry, configuration. Understanding: duplicated checks, one copy reads easily; finding every copy does not; one small policy function, one place to read; a generic rule framework, indirect: follow the configuration to find a rule. Extension: duplicated checks, edit every copy; one small policy function, edit one function, recheck its callers; a generic rule framework, add a rule as data, if the framework supports its shape. Debugging: duplicated checks, copies can disagree, as in scene 2; one small policy function, one decision to inspect; a generic rule framework, behavior spreads across the engine and its configuration. Qualitative judgments for this example; no costs are measured.

Principle · Abstraction should earn its complexity by simplifying real responsibilities or plausible change.

In short · Add structure only when it pays for itself.

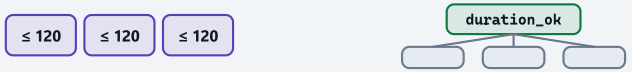
A flexible design can still be the wrong amount of design

Panel 2 of 3 · Several independently changing policies: the evidence that justifies more structure

Condition: **Changed: several policies that change independently** State: **4 of 4**

More structure can now pay for itself.

HYPOTHETICAL POLICIES FOR THIS COMPARISON
A shorter limit in exam weeks · Longer bookings for project rooms · Different booking hours per building



	Duplicated checks	One small policy function	A generic rule framework
SETUP	None	Small	Large
UNDERSTANDING	Many copies of many rules	One function grows long branches	Each rule reads as data
EXTENSION	Every policy times every copy	Every new policy edits the same function	A new policy becomes a new rule entry
DEBUGGING	Many places to disagree	Branches interact	Needs a way to see which rule decided
FITS?	does not fit: every copy multiplies	**start here: a policy table read by one function**	**grow toward it for rule shapes that really repeat**

Evidence for more structure: several policies change often and independently; the same rule shape repeats; people outside the code must change rules safely.

Hypothetical policies for this comparison: a shorter limit in exam weeks; longer bookings for project rooms; different booking hours per building. Evidence for more structure: several policies change often and independently; the same rule shape repeats; people outside the code must change rules safely. More structure can now pay for itself: start with a small policy table read by one function, and grow toward a framework only for rule shapes that really repeat.

Principle · Abstraction should earn its complexity by simplifying real responsibilities or plausible change.

In short · Add structure only when it pays for itself.

A flexible design can still be the wrong amount of design

Panel 3 of 3 · Build, reuse or buy the email capability

Condition: **Changed: the email capability could be built, reused or bought** State: **5 of 5**

Buying or reusing changes who does the work, not what you must check.

	 **Build a small adapter**	 **Reuse a maintained library**	 **Buy a managed capability**
INTEGRATION	written to the Notifier contract	wrap it in an adapter that keeps the contract	an adapter to its API, which changes on the provider's schedule
LICENSING	your own code	check that its license fits your use	a subscription and its terms
SUPPORT	your team	its maintainers and your team	the provider, under its terms
DATA PORTABILITY	full: your data stays in your systems	your data stays with you	check how addresses and delivery history can be exported
EXIT COST	low	moderate: replace the library behind the adapter	can be high: contract, data export, a new adapter

Still required: Whichever you choose, the Notifier contract checks of scene 4 still run.

Build a small adapter: integration, written to the Notifier contract; licensing, your own code; support, your team; data portability, full: your data stays in your systems; exit cost, low. Reuse a maintained library: integration, wrap it in an adapter that keeps the contract; licensing, check that its license fits your use; support, its maintainers and your team; data portability, your data stays with you; exit cost, moderate: replace the library behind the adapter. Buy a managed capability: integration, an adapter to its API, which changes on the provider's schedule; licensing, a subscription and its terms; support, the provider, under its terms; data portability, check how addresses and delivery history can be exported; exit cost, can be high: contract, data export, a new adapter. Whichever you choose, the Notifier contract checks of scene 4 still run.

Principle · Abstraction should earn its complexity by simplifying real responsibilities or plausible change.

In short · Add structure only when it pays for itself.

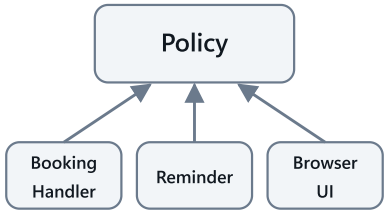
A future maintainer needs the reason, not only the diagram

Panel 1 of 2 · The decision record behind “Use a Policy component”

Condition: **Baseline: one campus, one set of booking rules** State: **6 of 6**

Revisit when: the conditions that would make this choice obsolete. Linked to R-01 and R-02.

WHAT THE MAINTAINER SEES



“Use a Policy component”

DR-001 **Use a Policy component for the booking rules** Accepted

CONTEXT

The R-02 duration rule was copied into the Browser UI, the Booking Handler and the Reminder. A missed copy gave inconsistent decisions, and an exam-week variant of 90 minutes is planned.

ALTERNATIVES CONSIDERED

- **Keep the copies and search before every change:** not chosen, a search for 120 missed the Reminder's copy, written 2 * 60.
- **A generic rule framework:** not chosen, two clear rules and a small team do not repay its setup and indirection.

DECISION

One Policy component owns R-01 and R-02. The Booking Handler and the Reminder ask it; the Browser UI reads its limits for early feedback and never decides.

CONSEQUENCES

- One place to edit a booking rule
- Policy checks and the handler's integration check run on every rule change
- Three components depend on Policy, so its interface changes carefully
- Scope: one set of booking rules for one campus. Not covered: per-room or per-campus rules, and a general rule engine.

REVISIT WHEN

- Campuses with different rules
- Staff need to change rules without a release

REQUIREMENTS R-01 R-02

DR-001, Use a Policy component for the booking rules (Accepted; R-01 and R-02). In practice, many teams keep such records as short Markdown files next to the code.

Principle · A design decision is easier to maintain when its reasoning and limits are recorded.

In short · Write down why, and when to rethink it.

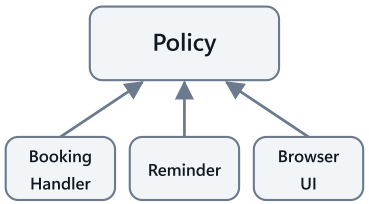
A future maintainer needs the reason, not only the diagram

Panel 2 of 2 · Several campuses: the revisit trigger fires

Condition: **Changed: several campuses, each with different booking limits** State: **4 of 4**

The record tells you when it has expired.

WHAT THE MAINTAINER SEES



"Use a Policy component"

New situation: Several campuses, each with different booking limits.

KEEP IT BLINDLY
Keep one Policy with one set of limits and patch each campus's exceptions into it.

REVISIT WITH THE TRIGGER
Reopen the record: weigh per-campus policy data, one Policy per campus, or a policy table, against the same context and consequences.

DR-001 **Use a Policy component for the booking rules** Revisit due

CONTEXT
The R-02 duration rule was copied into the Browser UI, the Booking Handler and the Reminder. A missed copy gave inconsistent decisions, and an exam-week variant of 90 minutes is planned.

ALTERNATIVES CONSIDERED
- **Keep the copies and search before every change:** not chosen, a search for 120 missed the Reminder's copy, written 2 * 60.
- **A generic rule framework:** not chosen, two clear rules and a small team do not repay its setup and indirection.

DECISION
One Policy component owns R-01 and R-02. The Booking Handler and the Reminder ask it; the Browser UI reads its limits for early feedback and never decides.

CONSEQUENCES
- One place to edit a booking rule
- Policy checks and the handler's integration check run on every rule change
- Three components depend on Policy, so its interface changes carefully
- Scope: one set of booking rules for one campus. Not covered: per-room or per-campus rules, and a general rule engine.

REVISIT WHEN
- ▶ **Campuses with different rules**
- Staff need to change rules without a release

REQUIREMENTS R-01 R-02

Several campuses, each with different booking limits. Trigger: Campuses with different rules. Keeping it blindly: keep one Policy with one set of limits and patch each campus's exceptions into it. Revisiting: reopen the record: weigh per-campus policy data, one Policy per campus, or a policy table, against the same context and consequences.

Principle · A design decision is easier to maintain when its reasoning and limits are recorded.

In short · Write down why, and when to rethink it.

Eight principles, six checks

One principle per scene, then six checks: decide your answer before you open each one.

Principles

1 · BRIDGE

Design quality becomes visible through the changes and failures a structure must accommodate.

In short: Structure shows itself when something changes.

`week-05.html#/same-behavior-different-design`

2 · ANCHOR

Useful boundaries localize decisions while keeping their dependents and evidence visible.

In short: Keep each decision in one place, and its users visible.

`week-05.html#/dependency-example`

3 · BRIDGE

Cohesion concerns meaningful related responsibility, not simply making modules small.

In short: Group what changes together; small alone is not the goal.

`week-05.html#/cohesion-and-responsibility`

4 · ANCHOR

An interface contract includes behavior and failure semantics, not just matching names or parameter lists.

In short: Matching names are not matching behavior.

`week-05.html#/interface-contract`

5 · BRIDGE

Depend on the behavior a responsibility needs, and isolate details that change for other reasons.

In short: Depend on what you need, not on how it is stored.

`week-05.html#/dependency-direction`

6 · ANCHOR

Refactoring should preserve intended observable behavior; small steps and relevant checks make deviations easier to locate.

In short: Change the structure in small steps; keep the answers.

`week-05.html#/refactoring-under-checks`

7 · BRIDGE

Abstraction should earn its complexity by simplifying real responsibilities or plausible change.

In short: Add structure only when it pays for itself.

`week-05.html#/abstraction-cost`

8 · BRIDGE

A design decision is easier to maintain when its reasoning and limits are recorded.

In short: Write down why, and when to rethink it.

`week-05.html#/design-rationale`

Checks: decide your answer before reading the one below each question

1. Why is a duplicated duration rule risky?

> Copies can change inconsistently and create conflicting decisions.

2. Does a centralized rule mean only one check is needed?

> No. Different checks should still independently examine its important behavior and integration.

3. Why did the replacement notifier break the design despite matching parameters?

> It violated the expected failure behavior and affected confirmation.

4. What should an architecture or design decision record preserve?

> Context, alternatives, choice, consequences, and the assumptions or triggers that warrant revisiting it.

5. Does a well-typed result prove that cancellation is authorized?

> No. Shape, domain validity and permission are separate claims with separate evidence.

6. An image library changes its file decoder while retaining the same function name. What does a caller still need to check?

> Check accepted inputs, pixel and color meaning, error results and side effects promised by the interface. A matching name or type shape does not settle behavioral compatibility.

What carries forward, and where to read more

Next week, these responsibilities cross process, network and data-consistency boundaries.

Week 6 starts from this question:

What changes when a dependency becomes slow, unavailable, or uncertain?

Carried forward to Week 6

- Module responsibilities: Booking Handler, Policy, Room Catalog, Notification and Storage.
- Contracts: Notifier.send with its failure behavior, and the three kinds of booking result.
- A decision record: DR-001, use a Policy component for the booking rules, with its revisit triggers.

In practice · examples from 2026

Editors such as VS Code and IntelliJ IDEA automate small refactorings like extracting a function, and many teams keep decision records as short Markdown files next to the code.

Sources for this lecture

S01 IEEE Computer Society, Guide to the Software Engineering Body of Knowledge (SWEBOK V4.0a) https://www.computer.org/education/bodies-of-knowledge/software-engineering

S02 ACM / IEEE-CS / AAAI, Computer Science Curricula 2023, final report https://csed.acm.org/final-report/

S07 Google Engineering Practices, What to look for in a code review https://google.github.io/eng-practices/review/reviewer/looking-for.html

S24 Titus Winters, Software Engineering at Google, chapter 1: What is software engineering? https://abseil.io/resources/swe-book/html/ch01.html

Terms used this week

Design
The internal structure: which part holds which decision, and which part depends on which.

Dependency
Code that needs another part to do its work; it may have to change when that part changes.

Coupling
How strongly parts depend on each other, for example through a copied decision.

Cohesion
How closely the contents of one module belong together, judged by their reason to change.

Responsibility
One reason to change that a module answers for.

Interface
What a part offers to its callers: its operations and their promises.

Contract
The promises of an interface: inputs, results, failure behavior and side effects.

Precondition
What must be true before a call, such as a stored booking id.

Encapsulation
Keeping a changeable detail, such as a row shape, inside one part.

Dependency direction
Which way source code depends; it can differ from the direction of runtime calls.

Adapter
A small part that converts one interface into another, such as storage rows into intervals.

Refactoring
A change of structure that keeps the intended observable behavior.

Regression
Behavior that used to be right and changed by mistake.

Abstraction
A simpler face over details; it costs setup and indirection.

Decision record
A short note of context, alternatives, decision, consequences and when to revisit.